

Inventario Belleza API

Documento Maestro del Proyecto

Spring Boot 3.2.0 | Java 17 | SQL Server | Swagger OpenAPI 3
JPA / Hibernate | JUnit 5 + Mockito | Maven | Perfiles dev/prod

Autor: Sebastian Rios
github.com/sebitasrios/inventario-belleza
2026

1. Guion de Exposicion

Deudas de la entrega anterior — lo que faltaba y como se corrigio

Diagrama ER

No estaba. Se agrego directamente en el README usando sintaxis Mermaid que GitHub renderiza nativamente. Mostrar: GitHub → README.

Perfiles

No estaban configurados. Se crearon application-dev.properties y application-prod.properties separando desarrollo y produccion. Mostrar: IntelliJ → resources → los 3 archivos properties.

Codigos de respuesta en controllers

Los metodos PUT y DELETE siempre retornaban 200 o 204 aunque el ID no existiera. Se agrego verificacion previa con obtenerPorId() y ahora retornan 404 correctamente. Mostrar: Swagger → PUT o DELETE con ID 999.

Clase SwaggerConfig

Ya existia pero no se habia mencionado explicitamente. Se documenta en el README. Mostrar: IntelliJ → config/SwaggerConfig.java.

Cierre de conexiones

Todos los DAOs usan try-with-resources que cierra automaticamente Connection, PreparedStatement y ResultSet. Mostrar: IntelliJ → dao/CategoriaDAOImpl.java.

Nuevas instrucciones — lo que se agrego

Pruebas unitarias

Se tienen 3 clases de prueba con JUnit 5 y Mockito: CategoriaServiceTest, ProductoServiceTest y ProveedorServiceTest. Para demostrarlo ejecutar en terminal: `.\mvnw test`

JPA + SQL

El proyecto usa dos estrategias en paralelo. SQL puro en los DAOImpl para CRUD, y JPA en los Repository para escenarios de negocio como buscar productos bajo stock, filtrar por categoria, buscar por email. Mostrar: Swagger → /productos/bajo-stock o /categorias/buscar?nombre=X

Desacoplamiento por interfaces

Los controllers nunca acceden directamente al service concreto ni al DAO. Siempre usan interfaces: ICategoriaService, IProductoService, IProveedorService. Mostrar: IntelliJ → cualquier controller, el @Autowired apunta a la interfaz.

Donde mostrar cada cosa al profesor

Que mostrar	Donde
Diagrama ER	GitHub → README

Perfiles dev/prod	IntelliJ → resources → 3 properties
Codigos 404 en PUT/DELETE	Swagger → PUT/DELETE con ID 999
SwaggerConfig	IntelliJ → config/SwaggerConfig.java
Cierre de conexiones	IntelliJ → dao/CategoriaDAOImpl.java
Pruebas unitarias	Terminal → .\mvnw test
JPA escenarios	Swagger → /productos/bajo-stock
Interfaces desacopladas	IntelliJ → cualquier controller

2. README — Documentacion Completa del Proyecto

Descripcion

Inventario Belleza es una API REST construida con Spring Boot que permite gestionar el inventario de productos de belleza. Implementa operaciones CRUD completas para productos, categorias y proveedores, con un enfoque dual de acceso a datos: SQL puro via JDBC para operaciones CRUD y JPA/Spring Data para escenarios de negocio.

Tecnologias

Tecnologia	Version	Uso
Java	17	Lenguaje principal
Spring Boot	3.2.0	Framework backend
Spring Web	-	Construccion de la API REST
Spring Data JPA	-	Acceso a datos con JPA/Hibernate
SQL Server	-	Base de datos relacional
SpringDoc OpenAPI	2.3.0	Documentacion Swagger UI
JUnit 5 + Mockito	-	Pruebas unitarias
Maven	-	Gestion de dependencias y build

Arquitectura

El proyecto implementa una arquitectura por capas con desacoplamiento mediante interfaces:

```
Controller → Interface Service → Service Impl → Interface DAO → DAO Impl (SQL)
→ JPA Repository (JPA)
```

- Los controllers acceden unicamente a interfaces de servicio (ICategoriaService, IProductoService, IProveedorService)
- Los services acceden al DAO para operaciones CRUD via SQL puro
- Los services acceden al Repository para escenarios de negocio via JPA
- Ningun controller accede directamente a un DAO o Repository

Estructura del proyecto

```
src/
main/
java/com/belleza/inventario/
config/ SwaggerConfig.java
controllers/ CategoriaController, ProductoController, ProveedorController
dao/ CategoriaDAO, ProductoDAO, ProveedorDAO (interfaces)
CategoriaDAOImpl, ProductoDAOImpl, ProveedorDAOImpl (SQL)
jpa/ CategoriaRepository, ProductoRepository, ProveedorRepository
entities/ Categoria, Producto, Proveedor
services/ ICategoriaService, IProductoService, IProveedorService
CategoriaService, ProductoService, ProveedorService
```

```

util/ ConexionDB.java
InventarioApplication.java
resources/
application.properties
application-dev.properties
application-prod.properties
test/
services/ CategoriaServiceTest, ProductoServiceTest, ProveedorServiceTest

```

Diagrama ER

El proyecto implementa 3 entidades con las siguientes relaciones:

Entidad	Campos principales	Relacion
CATEGORIA	id_categoria (PK), nombre	1 categoria → N productos
PROVEEDOR	id_proveedor (PK), nombre, telefono, email	1 proveedor → N productos
PRODUCTO	id_producto (PK), nombre, descripcion, precio, stock, stock_minimo	N productos → 1 categoria (FK) N productos → 1 proveedor (FK)

Base de datos — Script SQL

```

CREATE DATABASE inventario_belleza;
GO
USE inventario_belleza;
GO

CREATE TABLE categoria (
id_categoria INT PRIMARY KEY IDENTITY(1,1),
nombre VARCHAR(100) NOT NULL
);

CREATE TABLE proveedor (
id_proveedor INT PRIMARY KEY IDENTITY(1,1),
nombre VARCHAR(100) NOT NULL,
telefono VARCHAR(20),
email VARCHAR(100)
);

CREATE TABLE producto (
id_producto INT PRIMARY KEY IDENTITY(1,1),
nombre VARCHAR(100) NOT NULL,
descripcion VARCHAR(255),
precio DECIMAL(10,2) NOT NULL,
stock INT NOT NULL DEFAULT 0,
stock_minimo INT DEFAULT 5,
id_categoria INT,
id_proveedor INT,
CONSTRAINT fk_producto_categoria
FOREIGN KEY (id_categoria) REFERENCES categoria(id_categoria),
CONSTRAINT fk_producto_proveedor
FOREIGN KEY (id_proveedor) REFERENCES proveedor(id_proveedor)
);

```

Configuracion y Perfiles

application.properties — configuracion base:

```
spring.application.name=inventario
spring.profiles.active=dev
springdoc.swagger-ui.path=/swagger-ui.html
```

application-dev.properties — desarrollo:

```
server.port=8080
server.servlet.context-path=/api
spring.datasource.url=jdbc:sqlserver://localhost:1433;databaseName=inventario_belleza;encrypt=false;integratedSecurity=true;
spring.datasource.driver-class-name=com.microsoft.sqlserver.jdbc.SQLServerDriver
spring.datasource.username=
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.SQLServerDialect
spring.jpa.hibernate.ddl-auto=none
spring.jpa.show-sql=true
spring.jpa.open-in-view=false
logging.level.root=INFO
logging.level.com.belleza=DEBUG
```

application-prod.properties — produccion:

```
server.port=80
server.servlet.context-path=/api
spring.datasource.url=jdbc:sqlserver://SERVIDOR-PROD:1433;databaseName=inventario_belleza;encrypt=true;integratedSecurity=false;
spring.datasource.driver-class-name=com.microsoft.sqlserver.jdbc.SQLServerDriver
spring.datasource.username=SA
spring.datasource.password=TU_PASSWORD
spring.jpa.database-platform=org.hibernate.dialect.SQLServerDialect
spring.jpa.hibernate.ddl-auto=none
spring.jpa.show-sql=false
spring.jpa.open-in-view=false
logging.level.root=ERROR
```

Acceso a Datos — SQL vs JPA

SQL puro (JDBC) — operaciones CRUD:

Operacion	Metodo
Listar todos	obtenerTodos()
Buscar por ID	obtenerPorId(int id)
Crear	crear(T entidad)
Actualizar	actualizar(T entidad)
Eliminar	eliminar(int id)

JPA / Spring Data — escenarios de negocio:

Entidad	Escenario	Metodo
Categoria	Buscar por nombre exacto	findByNombre(String nombre)
Categoria	Buscar por nombre parcial	findByNombreContaining(String nombre)
Producto	Productos bajo stock minimo	findProductosBajoStockMinimo()
Producto	Filtrar por categoria	findByIdCategoria(int id)
Producto	Filtrar por proveedor	findByIdProveedor(int id)
Producto	Precio mayor a valor	findByPrecioGreaterThan(double precio)
Proveedor	Buscar por email	findByEmail(String email)
Proveedor	Buscar por nombre parcial	findByNombreContaining(String nombre)

Endpoints Disponibles

Base URL: <http://localhost:8080/api>

Categorías:

Metodo	Endpoint	Descripcion	Respuesta
GET	/categorias	Listar todas	200 OK
GET	/categorias/{id}	Buscar por ID	200 / 404
POST	/categorias	Crear	201 / 409
PUT	/categorias/{id}	Actualizar	200 / 404
DELETE	/categorias/{id}	Eliminar	204 / 404
GET	/categorias/buscar?nombre=	Buscar parcial (JPA)	200 OK

Productos:

Metodo	Endpoint	Descripcion	Respuesta
GET	/productos	Listar todos	200 OK
GET	/productos/{id}	Buscar por ID	200 / 404
POST	/productos	Crear	201 Created
PUT	/productos/{id}	Actualizar	200 / 404

DELETE	/productos/{id}	Eliminar	204 / 404
GET	/productos/bajo-stock	Bajo stock (JPA)	200 OK
GET	/productos/por-categoria/{id}	Por categoria (JPA)	200 OK
GET	/productos/por-proveedor/{id}	Por proveedor (JPA)	200 OK
GET	/productos/precio-mayor/{precio}	Precio mayor (JPA)	200 OK

Proveedores:

Metodo	Endpoint	Descripcion	Respuesta
GET	/proveedores	Listar todos	200 OK
GET	/proveedores/{id}	Buscar por ID	200 / 404
POST	/proveedores	Crear	201 / 409
PUT	/proveedores/{id}	Actualizar	200 / 404
DELETE	/proveedores/{id}	Eliminar	204 / 404
GET	/proveedores/buscar?nombre=	Buscar parcial (JPA)	200 OK
GET	/proveedores/por-email?email=	Por email (JPA)	200 / 404

Pruebas Unitarias

El proyecto incluye pruebas unitarias con JUnit 5 y Mockito para la capa de servicios. Se prueban tanto los metodos CRUD (via mock del DAO) como los escenarios JPA (via mock del Repository).

Clase de prueba	Pruebas	Que cubre
CategoriaServiceTest	6	CRUD via DAO mock + JPA via Repository mock
ProductoServiceTest	9	CRUD via DAO mock + 4 escenarios JPA
ProveedorServiceTest	9	CRUD via DAO mock + buscar por email y nombre

Para ejecutar las pruebas:

```
.\mvnw test
```

Documentacion Swagger

Con la aplicacion corriendo, accede en:

```
http://localhost:8080/api/swagger-ui/index.html
```

La configuracion de Swagger se encuentra en SwaggerConfig.java dentro del paquete config.

Como Ejecutar

Prerrequisitos: Java 17+, SQL Server activo con la base de datos inventario_belleza creada.

```
# Clonar el repositorio
git clone https://github.com/sebitasrios/inventario-belleza.git
cd inventario-belleza

# Ejecutar en modo desarrollo
.\mvnw spring-boot:run "-Dfile.encoding=UTF-8"
```

Autor: Sebastian Rios — github.com/sebitasrios

Desarrollado con Spring Boot · 2026